

DWH SQL Drafting Assistant

POC / MVP architecture
and Elasticsearch metadata model

From business questions to reviewable SQL.

LOCAL INFERENCE

One A100

RETRIEVAL

Elasticsearch

METADATA

PowerDesigner + Accurity

Agreed implementation scope: a small SQL drafting application.
The broader architecture is retained as a reference appendix.

Reading guide

Build the minimal solution first. The broader architecture is reference material, not additional POC scope.

This document consolidates the agreed SQL drafting solution, its Elasticsearch data model, complete mapping and sample metadata. All generation stays on premises; users review and execute SQL through their existing tools.

Part	What it contains
1 - POC / MVP	Components, workflows, algorithms, contracts, deployment and acceptance
2 - Metadata model	Document boundaries, fields, payloads, retrieval and ingestion checks
3 - Reference architecture	The original expanded design, retained for future requirements
Appendix A	Complete Elasticsearch index creation body
Appendix B	Complete illustrative metadata documents and sample SQL

Configuration notes

The DWH engine and A100 memory capacity remain to be confirmed. Oracle SQL and 768 vector dimensions in the sample files are illustrative. Real embeddings must be generated before hybrid-search publication. No live Elasticsearch deployment or DWH validation has been performed.

Using this PDF

The contents and document outline are clickable. Internal file references jump to the relevant part or appendix. Official documentation links open their source pages. The original Markdown and JSON files are embedded as PDF attachments for exact reuse; long code lines in the printed appendices wrap for readability.

Source package

minimal-logical-architecture.md
elasticsearch-metadata-model.md
logical-architecture.md
elasticsearch-index-mapping.json
elasticsearch-sample-documents.json

Contents

Reading guide	2
Part 1 The minimal POC / MVP	4
1. Logical architecture	4
2. Components and responsibilities	5
3. The minimum knowledge package	5
4. Data flow A: prepare the knowledge	6
5. Data flow B: generate a SQL draft	7
6. Algorithms inside the backend	7
7. Minimal request and response contracts	8
8. Minimal user interface	9
9. Deployment, errors and POC-to-MVP changes	9
10. Acceptance and implementation order	10
Technical references	10
Part 2 Elasticsearch metadata model	11
1. Deliverables and assumptions	11
2. Document boundaries and logical relationships	11
3. Common document envelope	12
4. Structured payloads	14
5. Indexed fields versus preserved content	15
6. Search text and embeddings	16
7. Runtime lookups using this model	16
8. Ingestion validation and index lifecycle	17
Part 3 Broader architecture reference	18
Appendix A Index mapping	30
Appendix B Sample documents	31
B1. Invoices - DWH.FACT_INVOICE	31
B2. Customers - DWH.DIM_CUSTOMER	33
B3. Invoiced revenue: definition, customer join and invoice dates	35
B4. Invoiced revenue by customer for a specified period	37

AGREED IMPLEMENTATION SCOPE

Part 1 | The minimal POC / MVP

Status: proposed POC / MVP design • 23 September 2026

Objective: help power users reach a correct DWH query faster by generating an editable SQL draft from their question, existing metadata, and approved SQL examples. Users review and execute the SQL through their existing database tools.

Scope: one business domain, one target DWH dialect, one local generative model on the A100, and Elasticsearch retrieval. PowerDesigner and Accurity metadata is already accessible in Oracle. The Oracle metadata database does not determine the target DWH dialect.

1. Logical architecture

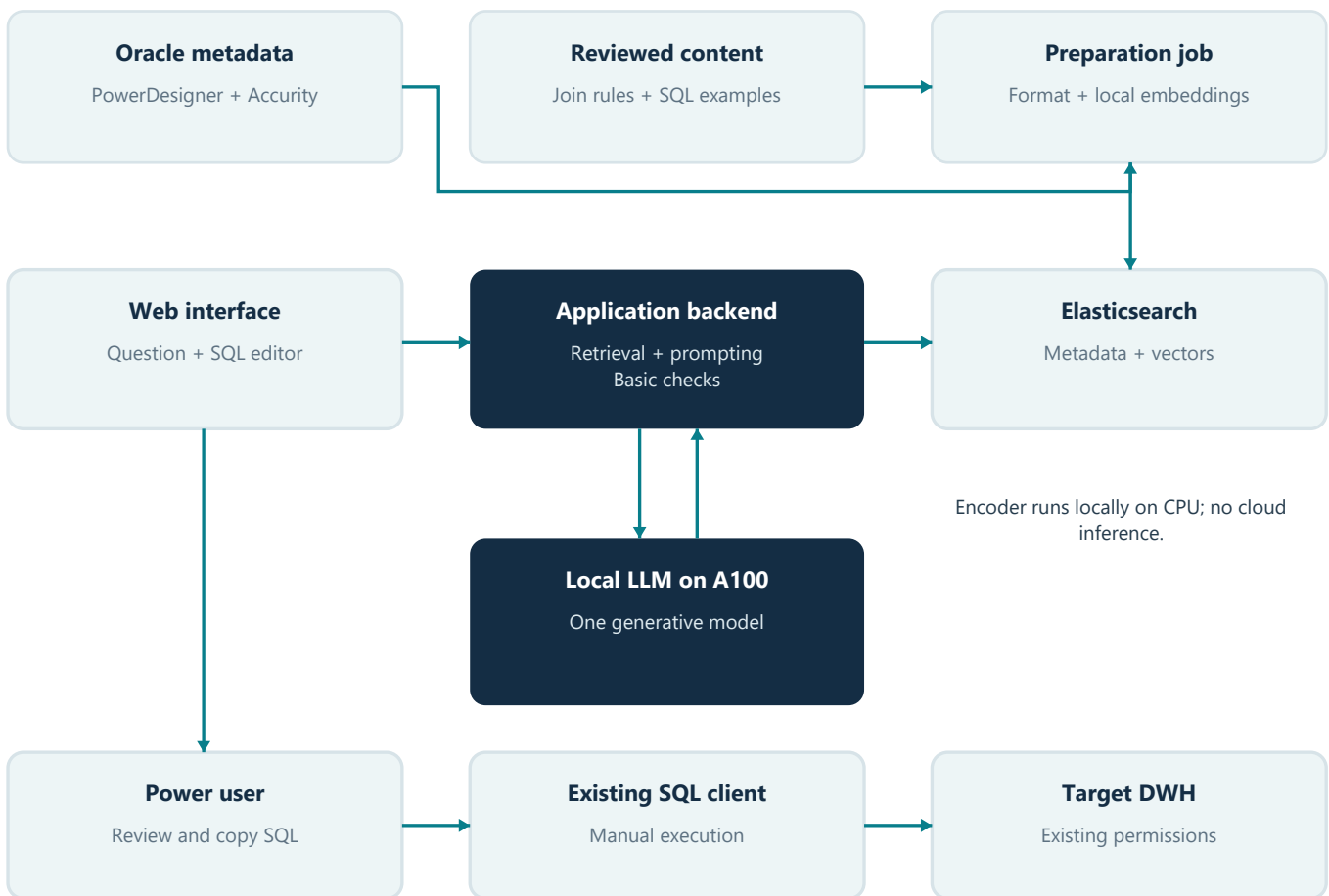


Figure 1. The agreed minimal architecture: four runtime components and a preparation job.

There are four runtime components: **web UI**, **application backend**, **Elasticsearch**, and **local model server**. The preparation script is a batch job. The embedding encoder is a shared local library/model loaded by the job and backend, not a separate service. Run the encoder on CPU initially and measure latency.

The backend is one application with ordinary functions. No agent framework, semantic compiler, graph database, independent planner, or automatic SQL execution service is required.

2. Components and responsibilities

Component	Responsibilities	Output
Oracle metadata source	Supply selected tables/views, columns, business definitions and mappings	Domain metadata snapshot
Approved content files	Supply table grain, join predicates, business/date rules and example queries	Small reviewed Markdown/JSON content set
Preparation script	Extract metadata, combine guidance, validate references, encode text and publish the index	Searchable domain knowledge
Elasticsearch	Store documents, keyword fields, vectors, types, domain IDs, dependencies and source references	Relevant context and exact dependency documents
Web UI	Accept questions; display SQL, clarification, assumptions and sources; support editing/copying	User requests and feedback
Backend	Apply configured domain/access scope; retrieve context; prompt model; perform basic checks	Draft response or actionable error
Local embedding encoder	Encode document text and questions using the same version/settings	Search vectors
Local model server	Host one generative model on A100; return SQL drafts or clarification	Structured response

Use an application configuration file for the selected domain, target dialect/version, model endpoint, encoder version, retrieval limits, context limits and timeouts. Connector credentials belong in protected configuration or the existing secret-management facility, never in prompts or indexed documents.

3. The minimum knowledge package

Start with one useful domain, approximately 10-20 tables/views, and 10-20 approved SQL examples. These are starting sizes, not fixed requirements. Prefer existing reporting views or data marts if they already implement the required business logic.

Index three document types in one logical Elasticsearch index:

Document type	Required content	Retrieval use
table	Fully qualified physical name; description; grain; columns/types; linked business terms and aliases; source links	Find physical objects from business language
guidance	Domain definitions; approved complete join predicates; required filters; date roles; current/historical behavior; code mappings where needed	Constrain how SQL is constructed
example	Business question; approved SQL; explanation; dialect; referenced table/guidance IDs	Adapt known working query patterns

Each document also contains `document_id`, `domain_id`, `document_type`, `content_version`, `source_references`, and `dependency IDs`. Physical names must be preserved exactly. Multiple mappings for one term remain visible rather than being silently collapsed.

The concrete model is defined in [Elasticsearch metadata model](#), with an [index mapping](#) and [four sample documents](#).

Stored content	Indexed search/filter fields	Preserved structured payload
Common envelope	Document type, domain, target connection/dialect, access scope, version, title, searchable text and embedding	Source references
Table/view	Physical identifiers, business-term IDs/names and aliases	Exact schema, columns/types, keys, grain and column-to-term mappings
Guidance	Searchable rule descriptions, dependency IDs and required-for-domain flag	Complete join predicates, business/date rules and code mappings
SQL example	Question/meaning in searchable text and dependency IDs	Reviewed SQL, parameter definitions, explanation and review reference

Business terms remain attached to their exact columns in the table payload. Root search fields are denormalized retrieval aids. The model uses one index and exact ID lookups, with no Elasticsearch join or separate term index. The mapping's vector dimension and the examples' Oracle SQL dialect are illustrative configurations to replace with actual deployment choices.

For wide tables, retrieve small searchable descriptions first and fetch the associated schema detail by exact document ID. Store complete guidance documents; do not split a join predicate across arbitrary text chunks.

Example guidance, to be replaced by actual approved rules:

```
Domain: Sales
Revenue: invoiced amount, excluding cancelled invoices.
Period: invoice date, using the calendar year.
Customer join: invoice.customer_key = customer.customer_key.
Customer interpretation: the customer version referenced by the invoice.
Output grain: identify customers by business key, not name alone.
```

The POC does not turn this prose into an executable rules engine. The model follows it, and the user reviews the resulting SQL. A DWH expert must supply missing rules rather than expecting the model to infer them.

4. Data flow A: prepare the knowledge

Run manually for the POC and on a schedule for the MVP:

1. Read the selected domain's metadata from Oracle.
2. Merge the reviewed join guidance and SQL examples by stable object IDs.
3. Check that referenced tables/columns exist in the supplied metadata. Flag missing targets and inconsistent definitions for the owner.
4. Build the three document types with source references and dependency IDs.
5. Generate embeddings locally using the same encoder that will encode user questions.
6. Build a new physical Elasticsearch index and run a small set of retrieval checks.
7. Switch the application's active index alias after successful completion. Keep the previous index available for in-flight requests and rollback.

A full rebuild of this small domain is the simplest starting algorithm. CDC, a separate semantic catalog, and incremental dependency services are unnecessary for the POC. Each request resolves the active alias once and uses that physical index for all its retrieval calls.

Show metadata refresh time in the UI. If refresh fails, keep the previous valid index and expose its age. A metadata snapshot can be stale relative to the DWH; the application does not claim live database validation.

5. Data flow B: generate a SQL draft

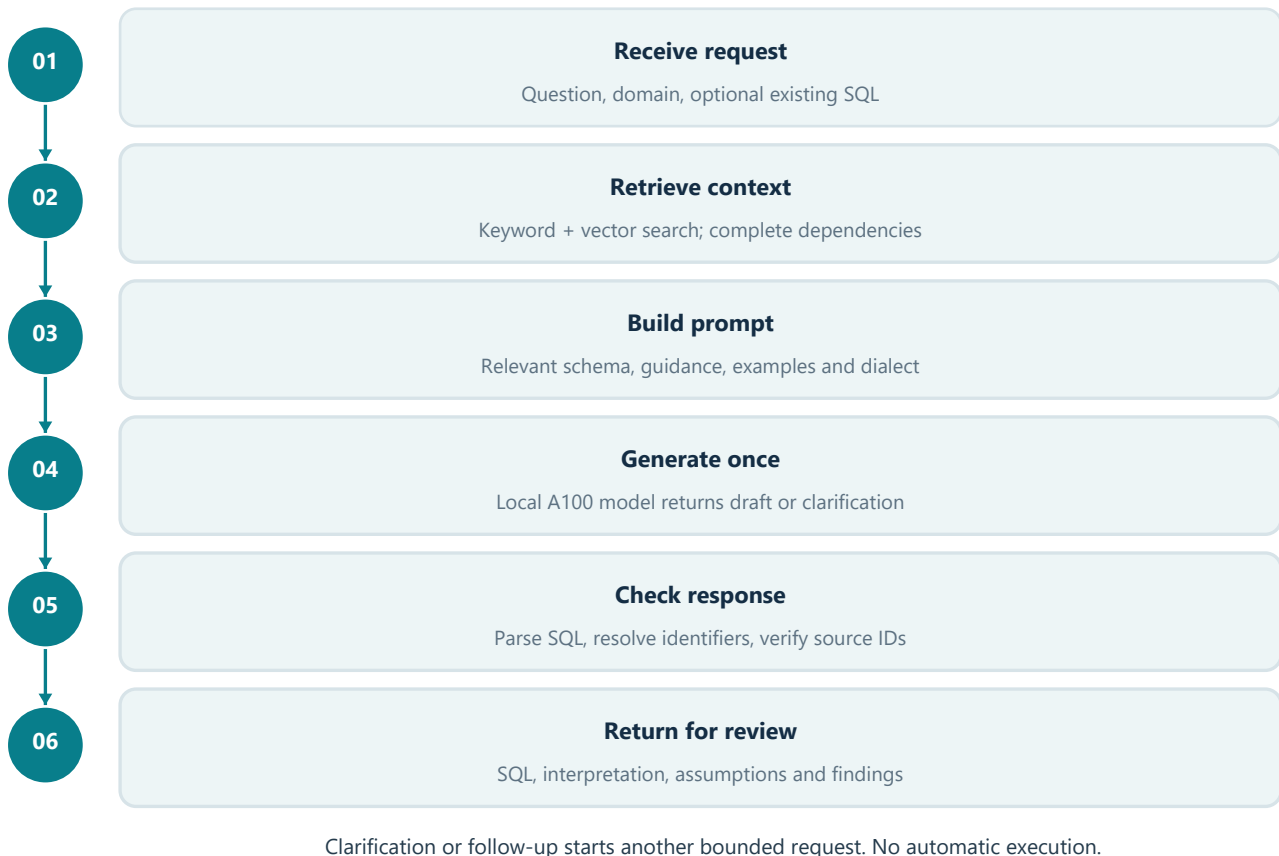


Figure 2. Minimal generation flow and user review.

No DWH query is executed in this flow. A normal request makes one generation call. Follow-ups repeat retrieval and checks using current metadata; the backend carries the previous draft and explicit decisions, not an unbounded conversation history.

6. Algorithms inside the backend

A. Retrieval. Filter by the configured domain and permitted metadata scope. Run BM25 keyword search over business terms, physical names and descriptions, and vector search over document embeddings. Combine rankings using reciprocal rank fusion:

$$\text{RRF}(\text{document}) = \text{sum}(1 / (k + \text{rank_in_each_list}))$$

Use a starting k of 60 and modest candidate lists, such as 20 hits per search branch. Preserve exact-name matches as candidates. Select a small set of relevant table documents and approximately three approved examples; tune these limits using local questions. RRF ranks relevance, not correctness.

B. Context completion. Fetch every schema document referenced by selected examples, plus the small domain guidance bundle. Include intermediate tables explicitly referenced by the guidance. This is an exact dependency lookup, not a graph search or join-planning algorithm. The curated domain bundle must include the relationships needed for supported questions.

C. Context budgeting. Count tokens with the deployed model's tokenizer. Reserve space for instructions and output. Remove low-ranked examples and optional descriptions first. If required schema and guidance cannot fit, ask the user to narrow the question; do not silently drop essential rules.

D. Generation. Ask the model to adapt an approved example when applicable, otherwise use the supplied schema and guidance. Require it to:

- Use the configured SQL dialect and supplied physical identifiers.

- Follow the documented joins, metric definitions and filters.
- Represent user-supplied values using named placeholders where supported, and list their values/types separately.
- Return a targeted clarification if an essential definition is missing or ambiguous.
- Produce one SQL draft, a short interpretation, assumptions and supporting document IDs.

Use structured output when the selected local server/model combination supports it. Otherwise parse and validate the response format in the backend. Low-temperature generation is a starting setting, not a guarantee of determinism or correctness.

E. Basic checks. Parse the SQL using a parser that supports the selected dialect. Check that it is a single supported read query, resolve aliases and CTE scopes, and compare physical table/column references against the supplied metadata. Unsupported parser constructs or unresolved references produce a visible finding rather than a false pass. Check returned source IDs against retrieved documents and generate links in application code.

These checks do not prove safe functions, correct joins, correct totals, current database permissions, runtime performance or business correctness. Required business rules remain guidance plus user review in this minimal version. Syntax failures return a revision-needed response; there is no autonomous repair loop.

7. Minimal request and response contracts

The backend supplies trusted identity/scope and resolves the target dialect from configuration. The client cannot broaden its access by naming another domain.

Request:

```
{
  "question": "Show invoiced revenue by customer for last calendar year",
  "domain_id": "sales",
  "previous_sql": null,
  "confirmed_decisions": []
}
```

The backend records a request ID, metadata index/version, model/prompt version, reference timestamp, and business timezone. Relative dates use this recorded context and documented calendar rules. A missing date role or calendar definition may require clarification.

Response:

```
{
  "status": "draft",
  "sql": "<generated SQL in the configured dialect>",
  "parameters": [],
  "interpretation": "Invoiced revenue grouped by customer business identifier.",
  "assumptions": [],
  "clarification_question": null,
  "sources": ["table.invoice", "table.customer", "guidance.sales"],
  "checks": {
    "syntax": "passed",
    "identifiers": "passed",
    "business_correctness": "not_verified",
    "database_execution": "not_run"
  }
}
```

Possible statuses: draft, needs_clarification, needs_revision, unsupported, and error. Check results are assigned by backend code, never accepted from model self-assessment. The example response describes the contract, not a real generated or validated query.

8. Minimal user interface

One page is sufficient:

- A fixed domain label, or an authorized domain selector if more than one is later enabled.
- A question box and Generate button.
- A SQL editor with copy and optional download actions.
- A short interpretation, assumptions, parameter values and source references.
- Individual check findings, clearly identifying the output as a draft.
- A follow-up box for changes or clarification answers.
- Useful / needs correction feedback, with optional corrected SQL.

Clarifications can be ordinary responses answered in the next request. No separate conversational platform is needed. If the user edits SQL directly, invalidate the previous check results until they request another check; never leave a stale passed indicator beside edited SQL.

Copy SQL and its parameter manifest together, or clearly display how to bind values in the user's client. Execution remains in the existing SQL client under existing DWH permissions.

9. Deployment, errors and POC-to-MVP changes

Deploy the UI and backend together if convenient. Run the preparation script on the same CPU host or existing scheduler. Reuse the existing Elasticsearch deployment. Host one generative model on the A100; choose its size and precision after confirming whether the card has 40 GB or 80 GB and measuring representative requests.

For a small POC, allow a bounded number of active generation requests and return a clear busy state. Add a simple bounded queue, timeout and cancellation behavior for the MVP. No second GPU model or external inference service is required by this design.

Condition	Behavior
No relevant schema or rule	Ask for a narrower question or report the missing information
Ambiguous business definition	Return one targeted clarification
Missing join guidance	Do not present an invented relationship as approved; return clarification/unsupported
Elasticsearch/model unavailable	Return a clear retryable error; preserve the user's question
Output cannot be parsed or identifiers cannot be resolved	Mark revision needed and show actionable findings
Metadata refresh fails	Retain last valid index and show metadata age
Direct SQL edit	Invalidate previous check status

For the POC, use a restricted internal deployment with a participant group authorized for the same domain metadata. If access differs between users, enforce authentication and per-user scope from the beginning. For MVP rollout, integrate existing SSO and server-side scope filtering; do not rely on UI filtering or prompts for permissions.

Keep minimal operational logs: request ID, versions, stage timings, error category and user feedback. Restrict and redact stored questions/SQL according to organizational policy. Do not log credentials or replicate DWH result rows. Feedback is a proposal: expert review is required before adding corrected SQL to the approved example library.

POC	MVP addition
Manual knowledge preparation	Scheduled refresh and failure visibility
One domain and small authorized cohort	Existing SSO and required metadata access controls
Generate, inspect and copy	Follow-up edits, feedback and polished error handling
Manually reviewed evaluation questions	Repeatable regression checks before changes
Single-request or limited-load serving	Bounded queue, measured capacity and operational monitoring

The logical architecture remains the same between POC and MVP.

10. Acceptance and implementation order

1. Select one domain and one DWH dialect; collect approximately 30-50 representative evaluation questions with expert-reviewed expected answers.
2. Assemble metadata, join guidance and approximately 10-20 approved SQL examples. Keep evaluation answers out of the retrieval example set.
3. Load the local model on A100 and create the Elasticsearch knowledge index.
4. Implement retrieval, prompt assembly, one generation call and the basic checks.
5. Add the single-page UI and run a pilot with 3-5 power users.
6. Compare time to a correct query with and without the assistant; record corrections, clarification burden, unsupported requests and latency.
7. Improve guidance/retrieval first, then consider model or prompt changes based on observed failures.

Agree numerical acceptance targets after a baseline measurement. SQL that parses or executes is not necessarily business-correct; domain experts assess the final interpretation and results using existing tools.

Deferred capabilities: automatic execution, result explanation, enterprise-wide domain coverage, executable semantic rules, automatic join planning, fine-tuning, autonomous repair, and query optimization. These are possible later extensions, not POC dependencies.

Technical references

- [Elasticsearch hybrid search](#): keyword/vector retrieval.
- [Elasticsearch reciprocal rank fusion](#): combining ranked result lists; use the installed version/license's supported interface, or fuse Elasticsearch result lists in the backend.
- [vLLM structured outputs](#): an available local-serving mechanism for constrained response structure if selected for implementation.

The broader design is retained in [logical-architecture.md](#) as a future reference. This minimal design is the implementation scope for the POC / MVP.

Part 2 | Elasticsearch metadata model

This model extends the minimal POC/MVP architecture. It uses **one index, three document types, and ordinary ID references**. There is no Elasticsearch parent/child join, separate graph, or additional metadata database.

The index is a retrieval copy of selected Oracle metadata plus expert-reviewed guidance/examples. Oracle and the reviewed source files remain authoritative.

1. Deliverables and assumptions

- **Index creation body** - explicit Elasticsearch field mappings.
- **Sample documents** - two tables, one guidance document and one SQL example.
- **Minimal architecture** - preparation and runtime flows.

The samples are fictitious and illustrate an Oracle 19c SQL target. This is not an assumption about the actual DWH. Replace the connection, dialect, schema, definitions, scopes and source references with real values.

The mapping uses **768 embedding dimensions as an illustrative configuration**. Set this to the chosen local encoder's actual dimension before creating the index. The sample documents intentionally omit vectors and embedding provenance: they can be indexed for lexical inspection but are not ready for a complete hybrid-search release. The loader must generate actual embeddings and populate the associated fields. Never substitute random or zero vectors.

Verify the mapping and search syntax against the installed Elasticsearch version before deployment. No index has been created by this documentation work.

2. Document boundaries and logical relationships

Type	One document represents	Structured payload
table	One physical table or view in the selected domain	table_metadata: physical identity, grain, keys, columns and their glossary mappings
guidance	One small coherent set of approved domain rules and join instructions	guidance: join definitions, metric/filter descriptions, date roles and code mappings
example	One reviewed question/query pattern	example: question, SQL, explanation, parameter definitions and review reference

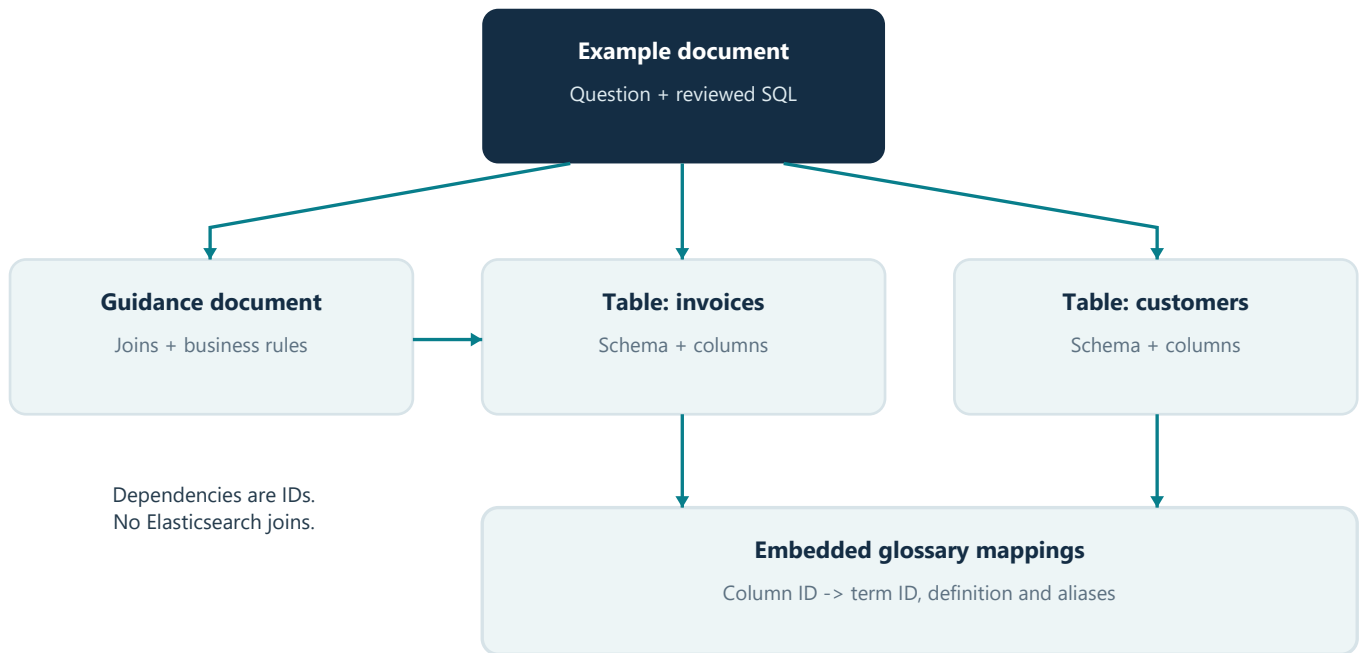


Figure 3. One index, three document types; glossary mappings remain attached to columns.

Arrows are ID references or embedded ownership, not Elasticsearch joins. The backend fetches dependent documents by ID with the same scope filters as search. Business terms are embedded with their mapped columns; one term can appear on multiple columns/tables. The authoritative column association is the structured payload, not the flattened search fields.

The POC indexes the selected domain, not automatically all 12,000 terms. A future term-only glossary browsing feature can introduce another document type, but SQL drafting does not require it.

3. Common document envelope

Elasticsearch arrays use the field's normal mapping: for example, `dependency_ids` is an array of keyword values, not a separate array type.

Field	Mapping	Population and purpose
<code>document_id</code>	keyword	Required stable ID; also use as Elasticsearch <code>_id</code>
<code>document_type</code>	keyword	Required enum: table, guidance, example
<code>domain_id</code>	keyword	Required selected business domain
<code>connection_id</code>	keyword	Required logical target DWH identifier; no credentials
<code>sql_dialect</code>	keyword	Required configured SQL engine name
<code>dialect_version</code>	keyword	Required configured target version/profile
<code>status</code>	keyword	Required approved for published POC content; loader governs admission
<code>access_scope_ids</code>	keyword[]	Required nonempty set of scopes permitted to see the entire document
<code>metadata_version</code>	keyword	Required snapshot/release identifier, identical across one published index
<code>content_version</code>	integer	Required positive version of this source content
<code>source_updated_at</code>	date	Latest known update across source records contributing to this document; omit if unknown

Field	Mapping	Population and purpose
indexed_at	date	Required preparation-job timestamp
title	text + exact keyword subfield	Required concise display/search title
search_text	text	Required full lexical search text assembled by the loader
physical_identifiers	keyword[]	Exact fully qualified physical identifiers; normally populated for tables
business_term_ids	keyword[]	Deduplicated Accurity IDs associated with the document
business_term_names	text[]	Denormalized names for boosted lexical search
aliases	text[]	Business synonyms and alternate labels for lexical search
dependency_ids	keyword[]	Required array; empty when no other documents are required
required_for_domain	boolean	Required; true only for guidance always included for the domain
source_references	Stored object array	Required provenance records: system, record ID, optional verified source URL
embedding_text	Stored, non-indexed text	Required for hybrid publication; exact text sent to the encoder
embedding_model_id	keyword	Required for hybrid publication; exact local encoder/checkpoint identity
embedding_pipeline_version	keyword	Required for hybrid publication; text construction/prefix settings version
embedding	dense_vector	Required for hybrid publication; finite, nonzero vector with configured dimension

Empty optional arrays are permitted. Exactly one of `table_metadata`, `guidance` or `example` must be populated, matching `document_type`.

`access_scope_ids` has OR semantics: possession of any listed scope allows reading the entire document. It must not be formed by taking the union of permissions for differently protected columns. Use one uniformly authorized domain for the POC. If visibility differs within a table, create appropriately scoped projections or enforce a more detailed policy before exposing documents to users or the model.

IDs should derive from stable source identifiers plus connection/domain/type, such as `table.dwh-poc.sales.pd-100`. Keep physical names as values, so a rename does not necessarily change the document's identity. A full rebuild removes deleted documents; other documents must not retain dangling dependencies.

4. Structured payloads

Table document

Field inside table_metadata	Content
object_id	Stable PowerDesigner/source object ID
schema_name, object_name	Exact physical names; add catalog name if the target engine requires it
object_type	table or view
description	Human-readable physical/business description
grain	What one row represents
primary_key_column_ids	Ordered source column IDs; support composite keys
unique_key_column_id_sets	Optional arrays describing other unique keys
columns	Complete structured column metadata for the supported table

Each column contains column_id, exact name, data_type, nullable, description, and business_terms. Use null/unknown for unavailable facts rather than fabricating constraints. Each mapped business term contains term_id, name, definition, aliases, and a source reference. Additional mapping-specific explanations can remain in this stored payload.

Conceptual example:

```
{
  "column_id": "pd-203",
  "name": "CUSTOMER_NAME",
  "data_type": "VARCHAR2(200)",
  "nullable": false,
  "description": "Current display name; not a unique customer identifier.",
  "business_terms": [
    {
      "term_id": "acc-customer-name",
      "name": "Customer name",
      "definition": "Name used to display the customer.",
      "aliases": ["client name", "account name"],
      "source_reference": {
        "system": "Accurity",
        "record_id": "acc-customer-name"
      }
    }
  ]
}
```

This is how the Accurity term-to-physical-column relationship is preserved. Search may return the table; application code then reads the exact matching column/term record from its payload.

Guidance document

Field inside guidance	Content
summary	What the guidance covers
table_ids	Referenced table document IDs
joins	Reviewed predicates, endpoint IDs, aliases, join type, cardinality and assumptions
rules	Approved metric expressions, mandatory filters and grouping explanations
date_role	Date column ID, calendar and interval convention
code_mappings	Column ID, stored code and business label

A join keeps the entire predicate together and, where available, the source column pairs. Composite joins need all key pairs. A predicate is prompt guidance; the minimal application does not execute it as a rule or automatically prove its correctness. Required runtime information belongs in `dependency_ids` as well as the explanatory payload.

For the small initial domain, use one required guidance bundle so missing business rules cannot be caused solely by a low retrieval score. Add optional guidance documents only when the bundle becomes unwieldy.

Example document

Field inside example	Content
question	Business question answered by this SQL pattern
sql	Complete reviewed SQL, preserved exactly with formatting
explanation	Why its joins, filters, dates and aggregations apply
parameters	Names, target types and descriptions of placeholders
table_ids, guidance_ids	Source documents required to adapt this example
review	Review reference and timestamp; add owner if available

Populate `dependency_ids` with the union of required table and guidance IDs. Changes to metadata require review/revalidation of affected examples before republishing them. Example SQL is instructional content for the LLM; the application still does not execute it.

5. Indexed fields versus preserved content

The JSON mapping sets `dynamic: strict` at the root. This rejects unexpected envelope fields and avoids accidental field proliferation. Required fields, enums, dependencies and payload structure still need ingestion validation; an Elasticsearch mapping is not a JSON Schema.

`table_metadata`, `guidance`, `example`, and `source_references` use `type: object`, `enabled: false`. Elasticsearch retains these payloads in `_source` without indexing their internal fields. This keeps the MVP simple and preserves column/term associations without requiring nested queries. Direct queries against `table_metadata.columns.business_terms` therefore will not work. Search the denormalized root fields, then inspect the payload in the backend.

Do not create one Elasticsearch field per physical column or glossary term. The field set stays fixed as the number of documents and columns increases. If exact within-column compound search becomes necessary later, revisit nested mappings or independent column documents then.

The mapping uses the standard text analyzer as a neutral baseline, with exact physical identifiers in keyword fields. Tune language handling and acronym/identifier tokenization using actual user questions. The loader can include readable, underscore-separated identifier variants in `search_text` while preserving physical spelling in the payload.

Official references: [dynamic mapping controls](#), [the enabled parameter](#), and [dense vectors](#).

6. Search text and embeddings

Construct lexical and embedding content deterministically:

Type	Lexical <code>search_text</code>	Suggested <code>embedding_text</code>
Table	Title, exact/readable names, grain, column names/types, business names/definitions/aliases	Coherent business description plus relevant column/term mappings
Guidance	Title, definitions, join roles, rule explanations and code labels	Business meaning, relationships and required interpretation
Example	Question, explanation, business concepts and object names	Question and explanation; raw SQL is preserved separately

Do not blindly concatenate long source documentation or ask the LLM to invent descriptions during ingestion. Use reviewed/source content. Keep lexical text complete enough to find relevant terms even if the encoder needs a shorter excerpt.

Check the encoder's input limit. Make any excerpting deliberate, store the exact `embedding_text`, and evaluate lost recall; do not silently truncate the encoder input. If one table is too wide to represent usefully, narrow the POC or introduce linked column-group retrieval documents as a later extension. This three-type version retains the complete table schema in its payload.

Use one encoder and preprocessing version for all documents and queries in a published index. If the encoder, input prefixes, dimension or text-construction contract changes, regenerate the corresponding document vectors. A different dimension requires a new index mapping. The backend obtains the query encoder configuration from the published index's deployment configuration, not from user input.

7. Runtime lookups using this model

Initial search: run lexical and vector searches with the same server-derived domain, connection, dialect/version, approval and access filters. Fuse ranks in Elasticsearch if supported, otherwise in backend code.

Illustrative lexical request body for the sample index:

```
{
  "size": 20,
  "_source": ["document_id", "document_type", "title", "dependency_ids", "required_for_domain",
    "metadata_version"],
  "query": {
    "bool": {
      "filter": [
        {"term": {"domain_id": "sales"}},
        {"term": {"connection_id": "dwh-poc"}},
        {"term": {"sql_dialect": "oracle"}},
        {"term": {"dialect_version": "19c"}},
        {"term": {"status": "approved"}},
        {"terms": {"access_scope_ids": ["sales-poc"]}}
      ],
      "must": [
        {"multi_match": {
          "query": "invoiced revenue by customer",
          "fields": ["title^3", "business_term_names^3", "aliases^2", "search_text"]
        }}
      ]
    }
  }
}
```

```

    }}
  ]
}
}
}
}

```

A separate exact-identifier clause can contribute candidates when the question names a known physical object. For vector search, encode the question and search embedding with those same filters; use the actual encoder output, not a placeholder array.

Required guidance: separately retrieve approved guidance documents with `required_for_domain: true` under the same filters, even if they were absent from the ranked hits.

Context completion: select table/example candidates, collect their `dependency_ids`, and fetch those IDs using a filtered `ids` query on the pinned physical index. Repeat only until references are complete, maintaining a visited set and small request/document budget. This is a bounded dependency expansion, not join-path inference. Missing or unauthorized required dependencies block use of the example/guidance rather than producing incomplete context.

An unrestricted `_get/_mget` does not apply the application's search filters. Prefer filtered ID lookups here, or explicitly enforce the same authorization on every direct lookup. Full payloads must never reach the model before scope checking.

Model context: read the complete `table_metadata`, `guidance` and `example` objects. Exclude embedding vectors and irrelevant index bookkeeping. Render readable schema and rules with their document IDs, allowing returned source references to be verified.

For "customer name," the root search fields identify the customer table; its payload identifies column `CUSTOMER_NAME` and term `acc-customer-name`. For "invoiced revenue by customer," the example and guidance add the invoice table, join predicate, currency convention and cancellation/date rules.

8. Ingestion validation and index lifecycle

Before publication, the preparation script must check:

1. Unique document IDs, `_id = document_id`, allowed types and exactly one matching payload.
2. Required envelope fields, current snapshot ID, approved source content and nonempty authorized scope.
3. Required table/column/term identities; primary/unique keys reference actual columns in that table.
4. Guidance table/column references and example dependencies exist in the same connection/domain/dialect scope.
5. Payload references and `dependency_ids` agree; no dangling dependencies or unexpected cycles.
6. Denormalized term IDs/names/aliases and physical identifiers are consistent with their source payloads. They are generated, never edited independently.
7. Each published hybrid-search document has valid embedding provenance and a finite, nonzero vector of the configured dimension.
8. A small retrieval check finds known terms and examples, and unauthorized scopes do not expose content.

Example physical index: `sql-assistant-metadata-poc-001`; application alias: `sql-assistant-metadata-current`. Use the mapping JSON as the body of index creation after selecting the encoder dimension. Load each object from the sample/data array as a separate document with its `document_id` as `_id`. The sample file is a JSON array, **not a Bulk API NDJSON request**.

Build a complete candidate index, check it, then switch the alias. Readers pin the physical index for all lookups in a request. This preserves the simple full-rebuild approach of the minimal architecture. Do not issue index creation/loading calls against a live cluster until connection, version, model and source metadata have been configured.

Part 3 | Broader architecture reference

The following is the original expanded architecture. Its semantic planner, compiler, execution broker and additional services are not required for the agreed minimal POC.

Design proposal • 23 September 2026

This architecture generates reviewable SQL for power users from business language, existing SQL, or both. Generative inference runs on one A100. Elasticsearch supplies retrieval. The existing Oracle metadata tables provide PowerDesigner and Accuracy metadata. The target DWH engine is deliberately unspecified: Oracle storing the metadata does not establish the SQL dialect of the target DWH.

Execution is an optional extension. The baseline product returns SQL, typed parameters, business interpretation, source references, assumptions, and validation findings. All components and data flows remain inside the organization. Component boundaries below are logical; they do not require separate microservices.

1. Architecture and responsibility boundaries

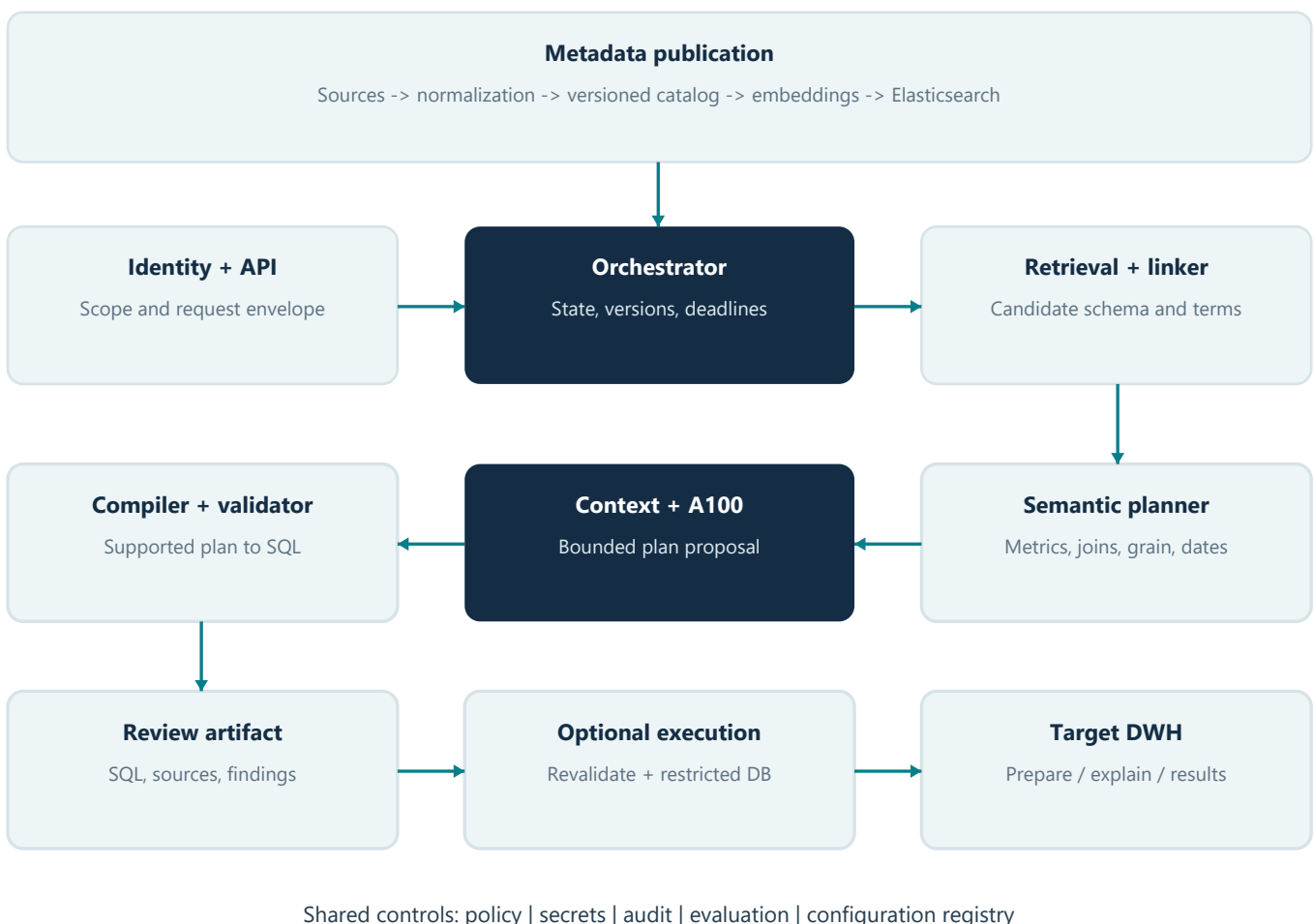


Figure 4. Broader reference architecture. Components and controls are detailed in the following table.

The model proposes an interpretation. Application code resolves catalog IDs, enforces permissions and semantic rules, and compiles supported plans. Elasticsearch is a search projection; it is not the authoritative source of metric formulas or join correctness. Retrieved text is data, never an instruction to grant access or execute tools.

2. Component responsibilities and contracts

ID	Component	Input → output	Implementation character
1	Snapshot/change reader	Oracle metadata and DWH catalog → staged source records	Scheduled delta reads or CDC when available; periodic full reconciliation
2	Normalizer/quality gate	Source records and curated rules → validated catalog records or quarantine findings	Stable IDs, type normalization, referential checks, source precedence
3	Semantic catalog	Published records → exact metadata by ID/version	Separate application schema in Oracle is sufficient; authoritative structured rules
4	Embedding service	Search text → embedding vector	Local encoder; same version and preprocessing for documents and requests
5	Elasticsearch	Query text/vector plus policy filters → ranked candidates	BM25, dense-vector search, rank fusion; no DWH fact replication required
6	Release manifest	Consistent catalog and index versions → immutable release ID	Atomic application-level publication pointer
7	UI/editor	User question, optional SQL and selected domain → review/clarification/edit interactions	Displays meaning, SQL, parameters, evidence and individual validation statuses
8	API/identity gateway	Authenticated request → trusted principal and request envelope	SSO integration; request limits; identity derived server-side
9	Orchestrator	Envelope and pinned versions → bounded workflow	Explicit state machine, deadlines, retries, cancellation and error classification
10	Session store	Request events and decisions → resumable state	Stores structured intent and revisions, not only a transcript
11	Retrieval/linker	Question and scope → candidate term/mapping/example IDs with evidence	Hybrid search, alias resolution, rank fusion and optional local reranking
12	Semantic planner	Candidate bindings and model proposal → resolved plan or clarification	Metric expansion, join planning, grain validation, ambiguity checks
13	Context builder	Relevant plan candidates/rules → bounded model input	Mandatory dependency closure, deduplication and token budgeting
14	Model gateway	Bounded inference task → structured output	Queue, token/context caps, timeout, model-version selection and schema validation
15	LLM server	Context and task schema → intent/plan/repair proposal	Local A100, one resident model initially
16	SQL compiler	Validated semantic plan → SQL AST, dialect SQL, typed bind parameters	Deterministic supported operators; engine-specific adapter

ID	Component	Input → output	Implementation character
17	Validator	SQL/AST, plan and policies → findings and stage statuses	Static SQL checks, semantic checks, optional DWH preparation/plan inspection
18	Artifact builder	Validated plan and SQL → review artifact	Sources and validation facts assembled from recorded evidence
19	Execution broker	Artifact revision/hash and parameters → bounded result page	Optional; current entitlements, DB restrictions, timeout and cancellation
20	Policy service	Principal and resource IDs → effective metadata and execution scope	Deny by default; application checks plus authoritative database enforcement
21	Audit/telemetry	Stage events → traces, operational metrics, restricted audit records	Redaction, retention and access controls
22	Evaluation/feedback	Reviewed outcomes and curated tasks → regression reports and proposed examples	Expert review before promotion; no automatic training on user edits
23	Configuration registry	Approved releases → pinned model/prompt/compiler settings	Rollback and reproducibility
24	Secret manager	Service identity → scoped connector credentials	No credentials in prompts, embeddings, UI artifacts or ordinary logs

3. Canonical semantic data model

Use stable identifiers independent of display labels. Preserve multiple physical mappings for a term rather than flattening them into one column. Each entity includes source reference, owner, status, effective dates, classification, version and content hash.

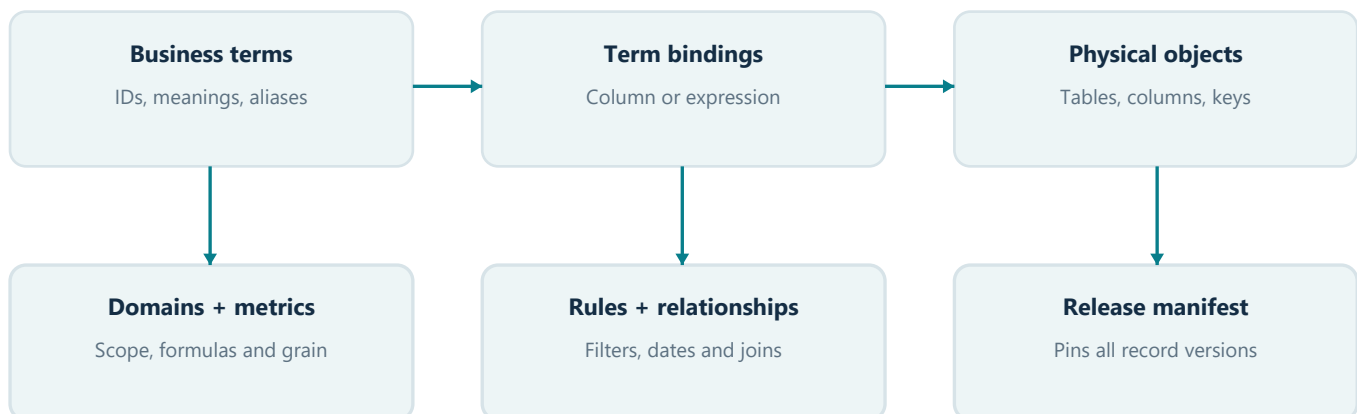
Entity	Essential fields
BusinessTerm	term_id, canonical_name, aliases, language, definition, domain_id, source_url
PhysicalObject	object_id, connection_id, catalog/schema/name, object_type, grain_key_ids, freshness information
Column	column_id, object_id, physical_name, data_type, nullability, key role, sensitivity
TermBinding	binding_id, term_id, target column or approved expression ID, source preference, domain applicability, semantic role, conditions
Relationship	edge_id, role-specific endpoints, composite key pairs or approved predicate AST, cardinality, uniqueness evidence, permitted direction/join type, temporal conditions, approval status
Metric	metric_id, definition, input bindings, expression AST, base grain, aggregation behavior by dimension, currency/unit, mandatory filters, date role, dependencies
Dimension	dimension_id, bindings, entity key, hierarchy, supported levels, historical/current behavior

Entity	Essential fields
BusinessRule	rule_id, applies_to IDs, predicate/expression AST, required/optional status, placement constraints, steward approval
ValueDictionary	column/binding ID, allowed code, business label, aliases, validity dates
QueryExample	example_id, question, dialect, domain, SQL AST/text, semantic plan, dependency IDs, approved metadata version
Domain	domain_id, scope description, authorized objects, supported capabilities, documented defaults
ReleaseManifest	release_id, catalog snapshot, physical index names, encoder version, schema fingerprint, publication status

Expressions must use a controlled expression representation or reviewed templates with typed placeholders. Descriptive prose is insufficient for executable metric definitions. Example dependency order: net-revenue metric → amount expression + cancellation rule + invoice date + currency conversion rule.

One many-to-one relationship with unverified uniqueness is not proof of safe aggregation. Record evidence as declared, profiled, or unknown; profiles can be stale. DWH owners must resolve missing or conflicting business rules.

Example logical relations:



Reference architecture only: these entities are not separate MVP services.

Figure 5. Conceptual entities for the broader reference architecture.

The ER diagram is conceptual. Bindings may also target expressions; relationship endpoints reference two role-specific object instances, and metric dependencies may reference other metrics.

4. Metadata publication flow and algorithms

- Acquire:** read source snapshots/deltas and assign an ingestion batch ID. For a watermark delta reader, use a consistent source snapshot or overlap window and stable IDs to avoid missing late commits. Reconcile deletions explicitly through tombstones or periodic full snapshots.
- Normalize:** retain source IDs; normalize names, types and relationships; hash canonical content. Do not merge terms solely because their labels match.
- Validate:** check binding targets, relationship endpoints, composite keys, formula dependencies, cycles and approval states. Compare published physical references with the live DWH catalog when access permits. Quarantine unusable

records and their dependent executable rules.

4. **Resolve source conflicts:** the DWH catalog governs physical existence/type; designated glossary owners govern business meaning; designated rule owners govern executable business rules. Conflicts become findings instead of silent overwrites.
5. **Build retrieval documents:** use logical term/binding, table, metric/rule and query-example records. Include domain, IDs, status, source references and access tags. Preserve a parent ID if long descriptions require splitting. Do not arbitrarily split join predicates or formulas.
6. **Embed:** calculate local vectors for changed searchable text only. Structured-only changes still update documents. Any embedding-model or preprocessing change requires a compatible reindex and query-encoder rollout.
7. **Build candidate release:** create versioned catalog data and physical Elasticsearch indexes; check counts, dangling dependencies, entitlement labels, deletion handling and retrieval smoke tests.
8. **Publish:** atomically update one release-manifest pointer only after all artifacts are ready. Readers pin its catalog version and physical index names. Old releases remain available for in-flight work; permission revocations still take effect immediately through live policy checks.
9. **Invalidate:** mark saved examples/artifacts affected by changed object/rule dependencies. Revalidate before reuse or execution.

An Elasticsearch alias can aid administration, but a request must not independently resolve a moving alias and a different catalog version. The pinned release manifest is the consistency boundary.

Suggested index families: `terms`, `schema`, `metrics-rules`, `examples`, each with a release suffix. Code dictionaries may be structured catalog lookups; large approved dictionaries can have their own searchable projection. Index metadata and approved labels, not unrestricted customer records.

5. Online request flow

Reference request sequence: identity and scope -> pinned versions -> retrieval -> semantic planning -> local model proposal -> semantic validation -> clarification or SQL compilation -> validation -> review.

Clarification resumes planning; the detailed sequence is also preserved in the attached source document.

Clarification resumes planning and validation; it does not bypass them. Ordinary requests need one generation call after retrieval. A separate intent-extraction call is optional for complex requests; retries are capped. All generative tasks use the same resident model initially.

6. Runtime algorithms

A. Scope and intent extraction

Resolve the target connection/dialect, principal, current permission version, selected domain, locale, business timezone and reference timestamp. The server supplies identity fields; user prompts cannot override them.

For simple input, deterministic tokenization, glossary aliases and date parsing can initiate retrieval directly. For complex input, ask the model for a constrained intent draft: measures, dimensions, filters, dates, desired grain, ordering and unknowns. Treat every draft field as a proposal. Do not use an uncertain domain classification as an irreversible filter: search several authorized domains or ask for a selection.

When modifying SQL, parse it first and resolve its identifiers into the same semantic representation where supported. Unsupported constructs remain a clearly marked advanced draft; they are not silently simplified.

B. Hybrid retrieval and entity linking

1. Normalize text with language-aware processing while preserving exact physical identifiers.
2. Generate the query vector locally using the release's encoder.

3. Run BM25 on boosted canonical names, aliases, physical names and descriptions; run vector search on the same authorized/versioned scope.

4. Fuse results using reciprocal rank fusion:

$$\text{score}(d) = \text{sum}(1 / (k + \text{rank}_i(d)))$$

Absent candidates contribute zero. $k = 60$ is a starting tuning value, not a confidence threshold.

5. Deduplicate by stable entity ID without losing alternative physical bindings. Optionally rerank the candidate union with a small local cross-encoder if CPU latency permits.

6. Link request phrases to term/metric IDs using exact alias evidence, retrieval ranks, role compatibility and domain compatibility. Preserve competing candidates.

7. Resolve catalog bindings by exact ID and expand all mandatory dependencies.

Starting experimental budgets: top 50 lexical + top 50 vector candidates per relevant search collection, optional reranking of up to 50 fused candidates, and 3-5 approved SQL examples after binding. These are tuning parameters; verify schema recall on the evaluation set. Permission and release filters apply to both search branches and again during dependency expansion.

Elasticsearch documents hybrid search and RRF in its [hybrid search guide](#) and [RRF reference](#). Use native fusion if supported by the installed version/license, or combine the two Elasticsearch result lists in application code.

C. Value and temporal resolution

Map labels such as "active" through approved code dictionaries; never invent stored codes. Resolve named entities through an authorized lookup service only if required and explicitly included in scope. Ambiguous entity matches require clarification.

Resolve relative dates from the recorded reference timestamp, business timezone and calendar. For "last year," use the documented calendar default or ask whether the user means calendar/fiscal year. Represent date filters as typed half-open intervals $[start, end)$ when appropriate. Require a semantic date role, such as invoice date or payment date, not merely a DATE-typed column.

D. Join planning

Maintain an in-memory adjacency view of the versioned relationship catalog. Nodes are role-specific table instances: billing customer and shipping customer remain distinct even if they use the same physical table. Edges carry complete key predicates, permitted join type/direction, cardinality, temporal rule and approval status.

Algorithm:

1. Expand metric dependencies and identify required base facts, dimensions and time roles.
2. Prefer approved domain query patterns or approved join subgraphs.
3. Otherwise enumerate bounded candidate paths between required nodes over eligible edges. Use nonnegative weighted shortest paths or bounded k-shortest-path search; combine candidates into a connected subgraph and prune redundant edges. This is a bounded heuristic, not a guarantee of the globally optimal join tree.
4. Rank feasible candidates by approved source preferences and lower structural complexity. Unknown temporal semantics, forbidden relationships and unsafe cardinality are hard constraints, not merely small penalties.
5. Add intermediate tables even when retrieval did not return them.
6. Validate the chosen subgraph against output grain, metric aggregation rules, join optionality, predicate placement and temporal behavior.
7. If multiple business-distinct paths remain, ask the user to choose. If no supported path exists, report the missing relationship.

Do not equate shortest path with correct business meaning. A purchase can join to the purchaser, payer or account owner through equally short paths.

E. Grain and aggregation validation

Represent the grain of each fact and intermediate relation as entity/key IDs. Track which joins preserve rows and which can multiply them. For a measure from a base fact, require proof or an approved rule that aggregation after joins preserves its intended meaning.

For multiple facts, aggregate each fact independently to an approved common grain and join those aggregates using conformed dimensions. A many-to-many bridge requires an explicit allocation/deduplication rule. Do not use `DISTINCT` or `SUM(DISTINCT amount)` as a generic repair for fan-out.

Metrics must describe additive, semi-additive or non-additive behavior. Ratios may require aggregating numerator and denominator separately; averages may require weights; balances may need an end-of-period rule. If the catalog lacks the relevant rule, ask or return an unsupported-plan finding.

F. Context assembly

Build a typed context package containing the authorized catalog slice, selected and competing bindings, complete join predicates, metric formulas, temporal/code rules, relevant examples, dialect/capability description, explicit user decisions and unresolved slots.

Budget rule:

```
context_budget = configured_context_limit - output_budget - instruction_tokens - safety_margin
```

Pack mandatory rules and dependency closure first. Add optional descriptions and examples by relevance within the remaining budget. Never truncate a required join predicate or omit a required filter merely to fit the prompt. If mandatory context cannot fit, narrow scope or return a clarification request. Token counts use the actual deployed model tokenizer.

G. Constrained plan generation

Use schema-constrained output where supported. Let the model choose among supplied catalog IDs and supported operators rather than invent physical identifiers. The output is a semantic plan proposal, not trusted executable text. It must represent unresolved decisions explicitly.

vLLM provides structured generation using JSON schemas and grammars; see its [structured output documentation](#). Valid structure does not establish valid business semantics, so application validation remains mandatory.

Initial supported operators: projection, typed filters, approved joins, grouping, approved metrics, ordering, limits and a small set of reviewed window/CTE patterns. Advanced operations can be added through compiler capabilities. Direct LLM SQL generation is an optional advanced-draft route; it must pass the same checks, and unprovable semantics remain visible and ineligible for automatic execution.

H. SQL compilation

Resolve plan IDs against the pinned catalog. Build a SQL abstract syntax tree (AST) from approved expressions and operators; apply mandatory predicates at their correct logical location; generate stable aliases and typed bind parameters; render through a target-engine adapter.

Do not string-concatenate user values into SQL. Identifiers come from authorized catalog bindings, since ordinary parameter binding does not bind table names. Outer-join predicates require particular care: moving a right-side condition into `WHERE` can change a left join's meaning.

The dialect adapter owns identifier quoting/case, date operations, parameter style, pagination, supported functions, null behavior and engine validation capabilities. Implement and verify one target dialect first.

I. Validation and bounded repair

Run checks in order:

Layer	Checks	Failure action
Contract	JSON schema, required slots, allowed operators, catalog ID membership	One structured repair or clarification

Layer	Checks	Failure action
Syntax/AST	Exactly one supported read query; inspect nested statements; allowed functions/constructs	Reject unsupported or potentially side-effecting constructs
Names/types	Resolve aliases, CTE scopes, columns, types and parameters	Repair binding/type error without changing the intended metric
Policy	Current object/column permission, classification, required row policy	Stop or re-scope; never "repair" by bypassing policy
Semantics	Grain, joins, metrics, dates, currency, nulls, output dimensions and rule coverage	Clarify missing meaning or reject unsupported plan
Database	Prepare/parse or non-executing plan inspection where supported	Classify live schema, syntax or resource findings
Resource envelope	Required bounded dates, estimated plan indicators, timeout/workload policy	Request narrower scope or decline execution

Database plan inspection is dialect-specific and may require auxiliary permissions or write a plan table. It must not use a command that executes the query, such as an execution-enabled analyze mode. Estimated costs are heuristics, not portable runtime guarantees. A read query can invoke side-effecting functions, so statement type alone is insufficient; use restricted database privileges and a controlled function surface.

Allow at most two repair attempts and a total deadline as initial policies. Give a repair task only the relevant redacted error, approved context and immutable business decisions. Revalidate from the beginning after a change. Detect repeated equivalent failures using a normalized plan/SQL fingerprint. Stop on unsupported semantics, policy failures or repeated errors.

J. Clarification and evidence-based status

Trigger clarification when multiple valid definitions materially alter results, required date/grain/currency information is missing, or business-distinct mappings remain unresolved. Present the alternatives in business language with their consequence; preserve the selected term/rule IDs in session state.

Do not treat retrieval scores or the model's reported confidence as a probability of correctness. Use explicit statuses such as `needs_clarification`, `draft`, `static_checks_passed`, `database_checks_passed`, `blocked` and `unsupported`. A skipped database check stays `not_run`; it is never reported as passed. An optional future risk score must be calibrated on labeled local evaluation data.

7. Data contracts

Each stage carries a request ID, revision and pinned release/config versions. Payloads below are illustrative; IDs, schema names and rules are fictitious.

RequestEnvelope

```
{
  "request_id": "req-001",
  "revision": 1,
  "principal_id": "server-resolved-user",
  "policy_version": "policy-42",
  "question": "Revenue by customer last year",
  "connection_id": "dwh-primary",
  "dialect": "configured-engine-version",
  "domain_ids": ["sales"],
  "locale": "en",
  "business_timezone": "Europe/Bucharest",
  "reference_timestamp": "2026-09-23T10:00:00+03:00",
}
```

```
"catalog_release": "catalog-017"
}
```

Resolved semantic plan, after the user has selected invoiced revenue, calendar year and customer at invoice time:

```
{
  "plan_version": "1",
  "catalog_release": "catalog-017",
  "metric_ids": ["metric.invoiced_revenue"],
  "dimension_ids": ["dimension.customer_at_invoice"],
  "grouping_key_ids": ["customer.business_id", "customer.name_at_invoice"],
  "time_filter": {
    "role_id": "date.invoice_date",
    "operator": "half_open_interval",
    "start_parameter": "p_start",
    "end_parameter": "p_end"
  },
  "parameters": {
    "p_start": {"type": "date", "value": "2025-01-01"},
    "p_end": {"type": "date", "value": "2026-01-01"}
  },
  "relationship_ids": ["join.invoice_customer_at_invoice"],
  "mandatory_rule_ids": ["rule.exclude_cancelled", "rule.reporting_currency"],
  "desired_grain": ["customer.business_id", "customer.name_at_invoice"],
  "unresolved_slots": []
}
```

This example deliberately exposes a grain choice: grouping by historical customer name can split one customer into several rows when the name changes. If the user wants exactly one row per customer, the display-name rule must be resolved separately. Grouping by name alone can merge different customers with identical names.

ReviewArtifact contains:

- Request/plan/artifact revision IDs; principal scope; catalog, model, prompt and compiler versions.
- SQL text, typed parameters, SQL/parameter hash and target connection/dialect.
- Business interpretation and explicit assumptions/clarifications.
- Output-column lineage to binding, term, metric and rule IDs; source links generated from catalog records.
- Validation results by layer, including passed, failed or not_run, plus timestamps.
- Unresolved limitations, execution eligibility and eligibility expiry/revalidation requirement.

Explanations are assembled from validated plan facts and provenance. An optional model paraphrase must not invent sources or make claims beyond recorded checks.

8. Execution and editing flow

The baseline is generate → inspect → edit/copy. Execution can be omitted without weakening SQL generation.

When enabled:

1. The user submits an explicit execution request referencing the exact artifact revision/hash and parameter values.
2. The broker rechecks current identity/entitlements, target schema compatibility and artifact validity. Editing SQL or values invalidates previous validation and requires new checks.
3. Establish a constrained database session using the user's identity or a verified delegated policy context; never a broadly privileged account without equivalent database-enforced restrictions.
4. Apply database statement timeout, workload/resource controls, fetch limits and cancellation. A result row limit is not a scan-cost limit.

5. Execute using typed binds. Return bounded/paginated results directly to the UI. The LLM does not need query results to generate SQL.
6. Record the execution ID, artifact hash, policy context, timing and outcome. Result rows are not placed in general telemetry or Elasticsearch by default.

Any later result-explanation feature is a separate authorized data flow with its own result-size and classification policy.

9. State, operational behavior and deployment

Request states:

```

RECEIVED -> AUTHORIZED -> RETRIEVING -> PLANNING
PLANNING -> NEEDS_CLARIFICATION -> PLANNING
PLANNING -> COMPILING -> VALIDATING -> READY_FOR_REVIEW
VALIDATING -> REPAIRING -> VALIDATING
READY_FOR_REVIEW -> EDITED -> VALIDATING
READY_FOR_REVIEW -> EXECUTION_REQUESTED -> REVALIDATING -> EXECUTING -> COMPLETED
Any applicable stage -> UNSUPPORTED | BLOCKED | FAILED | CANCELLED

```

Transitions have typed error reasons and bounded attempt counts. An execution failure does not silently trigger broader access or altered business semantics.

Deployment area	Logical components
Existing systems	Oracle metadata, target DWH, identity provider, PowerDesigner and Accurity integrations
CPU application service	API, orchestrator, retrieval, planner, compiler, validation, artifact construction, session management
CPU background worker	Metadata publication, embedding jobs, evaluations and curated-example maintenance
Elasticsearch cluster	Metadata, semantic and example retrieval
A100 inference service	One resident generative model behind a gateway and bounded admission queue
Shared platform services	Secrets, policy integration, telemetry, configuration/artifact storage

Use a local small embedding encoder and optional cross-encoder on CPU if they meet measured latency. These are encoders, not a second generative LLM. If they must share the A100, benchmark reserved capacity or schedule offline embedding work so it does not disrupt interactive generation. Do not swap large models for each workflow step.

Configure A100 memory based on weights + runtime buffers + KV cache at the chosen context/concurrency. A 40 GB card suggests evaluating a smaller BF16 model or a quantized 32B model; an 80 GB card provides more options. vLLM's [quantization compatibility table](#) lists supported Ampere paths. Actual model/quantization selection requires local SQL evaluation and load testing.

Use admission control with bounded active requests, context/output limits, cancellation and fair queuing. Treat the proposed retrieval budgets, repair counts and model sizes as experimental settings. A single A100 cannot supply redundant inference availability; if it fails, browsing metadata and reviewing saved SQL can remain available while new generation is unavailable.

Start with a modular application and worker rather than deploying every box separately. Separate model serving and Elasticsearch because their resource and lifecycle needs differ.

Cache keys include catalog release, model/prompt/compiler versions, target dialect, normalized request/decisions, principal or entitlement fingerprint, and relevant time context. Recheck current permissions on cache access. Do not reuse result data across principals through a generation cache.

Failure	Required behavior
Metadata release build fails	Keep last valid release; report publication failure and freshness
Current permissions cannot be verified	Stop protected retrieval/execution
Elasticsearch unavailable	Report retrieval unavailable; do not improvise unseen schema
Model unavailable or queue full	Return retryable status; keep editor/catalog browsing usable
Mandatory rule is absent	Ask for clarification or report unsupported definition
DWH validation unavailable	Return draft with database checks explicitly not run
Live schema has drifted	Invalidate affected artifact, refresh metadata and regenerate/revalidate
Repair budget exhausted	Return actionable findings; do not label the query validated

10. Evaluation, maintenance and release algorithm

Create an expert-reviewed dataset from actual power-user tasks, split by business scenario rather than superficial wording. Keep held-out tasks and paraphrases out of retrieved example sets where they would leak answers. Use fixed database snapshots or controlled fixtures for result comparisons, together with expert semantic review and edge-case data: accidental equality on one dataset does not prove equivalence.

Measure each layer separately:

Layer	Metrics
Retrieval	Recall of required terms, bindings, tables and rules; unauthorized exposure rate
Planning	Correct metric/date/grain selection, join validity, ambiguity detection and clarification burden
SQL	Syntax/type validity, execution success, correct results and business interpretation
User outcome	Time to accepted query, extent of edits, acceptance/rejection reasons
Serving	Queue wait, generation latency, total latency, throughput, GPU memory and error rate
Policy	Permission regression cases, nested construct rejection, row/column enforcement and revoked-access behavior

Release sequence: candidate model/prompt/catalog/compiler → offline regression → expert review of changed behavior → small pilot → monitored rollout → rollback if predefined acceptance gates fail. Establish those gates with stakeholders after baseline measurements; do not assume an accuracy percentage in advance.

Store user corrections as proposals. A steward approves business definitions, joins and examples before they become authoritative retrieval material. Invalid or outdated examples are removed from eligible retrieval when their dependencies change.

11. Implementation boundary and open decisions

First implementation: one DWH dialect, one or two domains, generate/edit/copy, structured metadata publication, hybrid retrieval, approved join paths, metric rules, constrained planning, deterministic SQL compilation and static validation. Add database validation where accessible. Add execution only if required by the product scope.

Decisions needed before physical sizing or detailed implementation:

1. A100 memory capacity, availability to this service, host RAM and deployment platform.
2. DWH engine/version, supported SQL surface, schema size and schema-change process.
3. Peak simultaneous requests, acceptable queue/response latency, availability expectations.
4. Existing coverage of join cardinality, grain, historical rules, metric formulas and approved SQL.
5. Elasticsearch version/license and existing authentication/authorization facilities.
6. User languages, business calendars/timezones and permitted metadata/value exposure.
7. Generation-only versus optional execution, and the required user-identity propagation model.

These affect configuration and integration choices. They do not change the central responsibility split: search retrieves evidence, the model proposes a plan, the semantic planner resolves business meaning, and deterministic code validates and compiles the supported SQL.

Appendix A | Index mapping

Complete index creation body. Set the illustrative 768-dimensional vector field to the selected encoder output size before index creation. Check compatibility with the installed Elasticsearch version.

```
{
  "mappings": {
    "dynamic": "strict",
    "_meta": {
      "purpose": "Minimal SQL assistant metadata",
      "contract_version": "1",
      "embedding_dimensions_note": "768 is illustrative. Set dims to the selected local encoder output dimension before index creation. Samples omit embeddings."
    },
    "properties": {
      "document_id": {"type": "keyword"},
      "document_type": {"type": "keyword"},
      "domain_id": {"type": "keyword"},
      "connection_id": {"type": "keyword"},
      "sql_dialect": {"type": "keyword"},
      "dialect_version": {"type": "keyword"},
      "status": {"type": "keyword"},
      "access_scope_ids": {"type": "keyword"},
      "metadata_version": {"type": "keyword"},
      "content_version": {"type": "integer"},
      "source_updated_at": {"type": "date"},
      "indexed_at": {"type": "date"},
      "title": {"type": "text", "fields": {"exact": {"type": "keyword"}}},
      "search_text": {"type": "text"},
      "physical_identifiers": {"type": "keyword"},
      "business_term_ids": {"type": "keyword"},
      "business_term_names": {"type": "text"},
      "aliases": {"type": "text"},
      "dependency_ids": {"type": "keyword"},
      "required_for_domain": {"type": "boolean"},
      "source_references": {"type": "object", "enabled": false},
      "table_metadata": {"type": "object", "enabled": false},
      "guidance": {"type": "object", "enabled": false},
      "example": {"type": "object", "enabled": false},
      "embedding_text": {"type": "text", "index": false},
      "embedding_model_id": {"type": "keyword"},
      "embedding_pipeline_version": {"type": "keyword"},
      "embedding": {"type": "dense_vector", "dims": 768, "index": true, "similarity": "cosine"}
    }
  }
}
```

Appendix B | Sample documents

Fictitious examples for an Oracle 19c target. Vectors are intentionally omitted. The embedded JSON attachment preserves the exact machine-readable document array.

B1. Invoices - DWH.FACT_INVOICE

Document type: **table**. ID: table.dwh-poc.sales.pd-100.

```
{
  "document_id": "table.dwh-poc.sales.pd-100",
  "document_type": "table",
  "domain_id": "sales",
  "connection_id": "dwh-poc",
  "sql_dialect": "oracle",
  "dialect_version": "19c",
  "status": "approved",
  "access_scope_ids": ["sales-poc"],
  "metadata_version": "poc-001",
  "content_version": 1,
  "source_updated_at": "2026-09-23T08:00:00Z",
  "indexed_at": "2026-09-23T09:00:00Z",
  "title": "Invoices - DWH.FACT_INVOICE",
  "search_text": "Invoices sales billed revenue invoice date customer invoice amount EUR euros
cancellation status. DWH.FACT_INVOICE: one row per invoice. INVOICE_ID invoice identifier;
CUSTOMER_KEY customer relationship; INVOICE_DATE invoice date; AMOUNT_EUR invoice amount in EUR;
INVOICE_STATUS invoice status. Consult approved revenue guidance before summing amounts.",
  "physical_identifiers": [
    "DWH.FACT_INVOICE",
    "DWH.FACT_INVOICE.INVOICE_ID",
    "DWH.FACT_INVOICE.CUSTOMER_KEY",
    "DWH.FACT_INVOICE.INVOICE_DATE",
    "DWH.FACT_INVOICE.AMOUNT_EUR",
    "DWH.FACT_INVOICE.INVOICE_STATUS"
  ],
  "business_term_ids": ["acc-invoice-date", "acc-invoice-amount"],
  "business_term_names": ["Invoice date", "Invoice amount"],
  "aliases": ["billing date", "billed amount"],
  "dependency_ids": [],
  "required_for_domain": false,
  "source_references": [{"system": "PowerDesigner", "record_id": "pd-100"}, {"system": "Accurity",
    "record_id": "sales"}],
  "table_metadata": {
    "object_id": "pd-100",
    "schema_name": "DWH",
    "object_name": "FACT_INVOICE",
    "object_type": "table",
    "description": "Illustrative invoice fact. AMOUNT_EUR is already expressed in EUR.",
    "grain": "One row per invoice.",
    "primary_key_column_ids": ["pd-101"],
    "columns": [
      {
        "column_id": "pd-101",
        "name": "INVOICE_ID",
        "data_type": "NUMBER(18)",
        "nullable": false,
        "description": "Unique invoice identifier.",

```

```

    "business_terms": []
  },
  {
    "column_id": "pd-102",
    "name": "CUSTOMER_KEY",
    "data_type": "NUMBER(18)",
    "nullable": false,
    "description": "Stable customer key referencing DIM_CUSTOMER.",
    "business_terms": []
  },
  {
    "column_id": "pd-103",
    "name": "INVOICE_DATE",
    "data_type": "DATE",
    "nullable": false,
    "description": "Invoice issue date.",
    "business_terms": [
      {
        "term_id": "acc-invoice-date",
        "name": "Invoice date",
        "definition": "Date on which the invoice is issued.",
        "aliases": ["billing date"],
        "source_reference": {"system": "Accurity", "record_id": "acc-invoice-date"}
      }
    ]
  },
  {
    "column_id": "pd-104",
    "name": "AMOUNT_EUR",
    "data_type": "NUMBER(18,2)",
    "nullable": false,
    "description": "Invoice amount already converted to EUR.",
    "business_terms": [
      {
        "term_id": "acc-invoice-amount",
        "name": "Invoice amount",
        "definition": "Amount of an invoice, expressed here in EUR.",
        "aliases": ["billed amount"],
        "source_reference": {"system": "Accurity", "record_id": "acc-invoice-amount"}
      }
    ]
  },
  {
    "column_id": "pd-105",
    "name": "INVOICE_STATUS",
    "data_type": "VARCHAR2(20)",
    "nullable": false,
    "description": "Non-null invoice status; cancellation code is documented in guidance.",
    "business_terms": []
  }
]
}
}

```

B2. Customers - DWH.DIM_CUSTOMER

Document type: **table**. ID: table.dwh-poc.sales.pd-200.

```
{
  "document_id": "table.dwh-poc.sales.pd-200",
  "document_type": "table",
  "domain_id": "sales",
  "connection_id": "dwh-poc",
  "sql_dialect": "oracle",
  "dialect_version": "19c",
  "status": "approved",
  "access_scope_ids": ["sales-poc"],
  "metadata_version": "poc-001",
  "content_version": 1,
  "source_updated_at": "2026-09-23T08:00:00Z",
  "indexed_at": "2026-09-23T09:00:00Z",
  "title": "Customers - DWH.DIM_CUSTOMER",
  "search_text": "Customers customer identifier customer name client name account name. DWH.DIM_CUSTOMER:
    one current row per customer, no historical versions in this illustrative model. CUSTOMER_KEY unique
    stable warehouse key; CUSTOMER_ID unique business identifier; CUSTOMER_NAME current display name, not
    necessarily unique.",
  "physical_identifiers": [
    "DWH.DIM_CUSTOMER",
    "DWH.DIM_CUSTOMER.CUSTOMER_KEY",
    "DWH.DIM_CUSTOMER.CUSTOMER_ID",
    "DWH.DIM_CUSTOMER.CUSTOMER_NAME"
  ],
  "business_term_ids": ["acc-customer-id", "acc-customer-name"],
  "business_term_names": ["Customer identifier", "Customer name"],
  "aliases": ["customer number", "client name", "account name"],
  "dependency_ids": [],
  "required_for_domain": false,
  "source_references": [{ "system": "PowerDesigner", "record_id": "pd-200" }, { "system": "Accurity",
    "record_id": "sales" }],
  "table_metadata": {
    "object_id": "pd-200",
    "schema_name": "DWH",
    "object_name": "DIM_CUSTOMER",
    "object_type": "table",
    "description": "Illustrative current customer dimension. CUSTOMER_KEY remains stable; there is no SCD
      history.",
    "grain": "One current row per customer.",
    "primary_key_column_ids": ["pd-201"],
    "unique_key_column_id_sets": [["pd-202"]],
    "columns": [
      {
        "column_id": "pd-201",
        "name": "CUSTOMER_KEY",
        "data_type": "NUMBER(18)",
        "nullable": false,
        "description": "Unique stable warehouse customer key.",
        "business_terms": []
      },
      {
        "column_id": "pd-202",
        "name": "CUSTOMER_ID",
        "data_type": "VARCHAR2(30)",
        "nullable": false,
        "description": "Unique business customer identifier.",
        "business_terms": [
```

```
{
  "term_id": "acc-customer-id",
  "name": "Customer identifier",
  "definition": "Business identifier distinguishing customers.",
  "aliases": ["customer number"],
  "source_reference": {"system": "Accurity", "record_id": "acc-customer-id"}
}
],
{
  "column_id": "pd-203",
  "name": "CUSTOMER_NAME",
  "data_type": "VARCHAR2(200)",
  "nullable": false,
  "description": "Current customer display name; not unique.",
  "business_terms": [
    {
      "term_id": "acc-customer-name",
      "name": "Customer name",
      "definition": "Name used to display the customer.",
      "aliases": ["client name", "account name"],
      "source_reference": {"system": "Accurity", "record_id": "acc-customer-name"}
    }
  ]
}
]
}
}
```

B3. Invoiced revenue: definition, customer join and invoice dates

Document type: **guidance**. ID: guidance.dwh-poc.sales.revenue.

```
{
  "document_id": "guidance.dwh-poc.sales.revenue",
  "document_type": "guidance",
  "domain_id": "sales",
  "connection_id": "dwh-poc",
  "sql_dialect": "oracle",
  "dialect_version": "19c",
  "status": "approved",
  "access_scope_ids": ["sales-poc"],
  "metadata_version": "poc-001",
  "content_version": 1,
  "source_updated_at": "2026-09-23T08:00:00Z",
  "indexed_at": "2026-09-23T09:00:00Z",
  "title": "Invoiced revenue: definition, customer join and invoice dates",
  "search_text": "Invoiced revenue billed revenue sales by customer in EUR. Sum invoice AMOUNT_EUR
    excluding CANCELLED invoices. Join invoices to customers using CUSTOMER_KEY. Filter invoice date with
    inclusive start and exclusive end. Group by CUSTOMER_ID and CUSTOMER_NAME. Calendar-year reporting;
    current customer name.",
  "physical_identifiers": [],
  "business_term_ids": ["acc-invoiced-revenue", "acc-customer-name", "acc-invoice-date"],
  "business_term_names": ["Invoiced revenue", "Customer name", "Invoice date"],
  "aliases": [],
  "dependency_ids": ["table.dwh-poc.sales.pd-100", "table.dwh-poc.sales.pd-200"],
  "required_for_domain": true,
  "source_references": [{"system": "Reviewed guidance", "record_id": "sales-revenue-v1"}],
  "guidance": {
    "summary": "Illustrative approved revenue rule for the two supplied tables.",
    "table_ids": ["table.dwh-poc.sales.pd-100", "table.dwh-poc.sales.pd-200"],
    "joins": [
      {
        "join_id": "invoice_customer",
        "left_table_id": "table.dwh-poc.sales.pd-100",
        "right_table_id": "table.dwh-poc.sales.pd-200",
        "left_alias": "i",
        "right_alias": "c",
        "join_type": "INNER",
        "cardinality": "many_to_one",
        "predicate_sql": "i.CUSTOMER_KEY = c.CUSTOMER_KEY",
        "key_pairs": [{"left_column_id": "pd-102", "right_column_id": "pd-201"}],
        "assumptions": [
          "DIM_CUSTOMER.CUSTOMER_KEY is unique.",
          "Each invoice has a matching customer.",
          "Customer keys are stable; no historical customer versions are present."
        ]
      }
    ]
  },
  "rules": [
    {
      "rule_id": "invoiced_revenue_eur",
      "description": "Sum AMOUNT_EUR; amounts are already in EUR.",
      "expression_sql": "SUM(i.AMOUNT_EUR)",
      "unit": "EUR"
    },
    {
      "rule_id": "exclude_cancelled",
      "description": "Exclude cancelled invoices; status is non-null in this example.",
      "predicate_sql": "i.INVOICE_STATUS <> 'CANCELLED'"
    }
  ]
}
```

```
    },
    {
      "rule_id": "invoice_period",
      "description": "Use invoice date, inclusive start and exclusive end.",
      "predicate_sql": "i.INVOICE_DATE >= :date_from AND i.INVOICE_DATE < :date_to"
    },
    {
      "rule_id": "customer_grouping",
      "description": "Group by customer business identifier and current name. Never group by name alone."
    }
  ],
  "date_role": {"column_id": "pd-103", "calendar": "calendar_year", "interval": "inclusive_start_exclusive_end"},
  "code_mappings": [{"column_id": "pd-105", "code": "CANCELLED", "label": "Cancelled"}]
}
```

B4. Invoiced revenue by customer for a specified period

Document type: **example**. ID: `example.dwh-poc.sales.revenue-by-customer`.

SQL displayed for review

```
SELECT c.CUSTOMER_ID,
       c.CUSTOMER_NAME,
       SUM(i.AMOUNT_EUR) AS INVOICED_REVENUE_EUR
FROM DWH.FACT_INVOICE i
JOIN DWH.DIM_CUSTOMER c
  ON i.CUSTOMER_KEY = c.CUSTOMER_KEY
WHERE i.INVOICE_STATUS <> 'CANCELLED'
      AND i.INVOICE_DATE >= :date_from
      AND i.INVOICE_DATE < :date_to
GROUP BY c.CUSTOMER_ID, c.CUSTOMER_NAME
```

Complete document

```
{
  "document_id": "example.dwh-poc.sales.revenue-by-customer",
  "document_type": "example",
  "domain_id": "sales",
  "connection_id": "dwh-poc",
  "sql_dialect": "oracle",
  "dialect_version": "19c",
  "status": "approved",
  "access_scope_ids": ["sales-poc"],
  "metadata_version": "poc-001",
  "content_version": 1,
  "source_updated_at": "2026-09-23T08:00:00Z",
  "indexed_at": "2026-09-23T09:00:00Z",
  "title": "Invoiced revenue by customer for a specified period",
  "search_text": "Show invoiced revenue in EUR by customer between two invoice dates. Billed revenue sales per client. Exclude cancelled invoices; group by unique customer business identifier and current customer name.",
  "physical_identifiers": [],
  "business_term_ids": ["acc-invoiced-revenue", "acc-customer-name", "acc-invoice-date"],
  "business_term_names": ["Invoiced revenue", "Customer name", "Invoice date"],
  "aliases": [],
  "dependency_ids": ["table.dwh-poc.sales.pd-100", "table.dwh-poc.sales.pd-200", "guidance.dwh-poc.sales.revenue"],
  "required_for_domain": false,
  "source_references": [{"system": "Reviewed SQL", "record_id": "sales-query-001"}],
  "example": {
    "question": "Show invoiced revenue in EUR by customer for a specified invoice-date period.",
    "sql": "SELECT c.CUSTOMER_ID,\n          c.CUSTOMER_NAME,\n          SUM(i.AMOUNT_EUR) AS\n          INVOICED_REVENUE_EUR\nFROM DWH.FACT_INVOICE i\nJOIN DWH.DIM_CUSTOMER c\n  ON i.CUSTOMER_KEY =\n  c.CUSTOMER_KEY\nWHERE i.INVOICE_STATUS <> 'CANCELLED'\n      AND i.INVOICE_DATE >= :date_from\n      AND i.INVOICE_DATE < :date_to\nGROUP BY c.CUSTOMER_ID, c.CUSTOMER_NAME",
    "explanation": "Uses the documented many-to-one customer join and invoice date. Amounts are already in EUR. Customers are grouped by identifier and current name.",
    "parameters": [
      {"name": "date_from", "data_type": "DATE", "description": "Inclusive invoice date."},
      {"name": "date_to", "data_type": "DATE", "description": "Exclusive invoice date."}
    ],
    "table_ids": ["table.dwh-poc.sales.pd-100", "table.dwh-poc.sales.pd-200"],
    "guidance_ids": ["guidance.dwh-poc.sales.revenue"],
    "review": {"review_reference": "illustrative-review-001", "reviewed_at": "2026-09-23T08:00:00Z"}
  }
}
```